Filtrage et détection de contours

Fiche de travaux pratiques • MATLAB / Python

Objectifs

Appliquer un filtre défini par ses coefficients, réduire le bruit et comparer les détecteurs Sobel, Prewitt et Canny.

Exercice 1 — Filtrage spatial avec un noyau

1. Lire l'image et l'afficher. Définir le filtre moyenneur H ci-dessous.

2. Appliquer H à I avec imfilter sous MATLAB ou cv2.filter2D sous Python.

3. Afficher côte à côte l'image originale et l'image filtrée.

4. Que deviennent les détails et les contours ?

5. Augmenter la taille du filtre à 9 × 9, puis à 11 × 11. Définir des noyaux moyenneurs dont les coefficients valent respectivement 1/81 et 1/121. Appliquer chaque filtre à l'image originale I.

6. Afficher et comparer les résultats des filtres 3 × 3, 9 × 9 et 11 × 11. Qu'observez-vous après augmentation de la taille du filtre ?

```
H = (1/9) × [ 1  1  1
              1  1  1
              1  1  1 ]
```

Exercice 2 — Bruit et filtrage

1. Ajouter un bruit « sel et poivre » à I pour obtenir une image bruitée Ib (densité 0,05).

2. Appliquer séparément un filtre moyenneur 3 × 3 et un filtre médian 3 × 3 à Ib.

3. Afficher l'original, l'image bruitée et les deux images filtrées.

4. Quel filtre élimine le mieux ce bruit tout en conservant les contours ?

Exercice 3 — Détection avec Sobel et Prewitt

Objectif

Détecter les contours d'une image et comparer deux méthodes.

1. Reprendre l'image originale I en niveaux de gris.

2. Appliquer Sobel dans les deux directions et obtenir la norme du gradient avec la fonction prédéfinie.

3. Faire de même avec Prewitt sur la même image.

4. Afficher côte à côte l'original, le résultat de Sobel et celui de Prewitt.

Exercice 4 — Détection avec Canny

Objectif

Détecter les contours avec Canny et comparer le résultat à Sobel.

1. Reprendre l'image originale I en niveaux de gris.

2. Appliquer Canny avec les paramètres par défaut.

3. Afficher côte à côte l'original, le résultat de Sobel et celui de Canny.

4. Avec quelle méthode les contours semblent-ils les plus fins et les plus continus ?

Dans les deux langages, Canny renvoie une carte binaire : contours blancs sur fond noir. Sobel fournit ici une intensité de gradient ; tenir compte de cette différence dans la comparaison.

Tableau des fonctions utiles

I désigne l'image originale réelle dans [0, 1], Ib l'image bruitée et H le noyau du filtre. Les fonctions ci-dessous servent à réaliser les quatre exercices.

Opération	MATLAB	Python
Lire une image	imread("cameraman.tif")	cv2.imread("cameraman.tif", 0)
Convertir dans [0, 1]	im2double(image)	img_as_float(image)
Afficher une image	imshow(I, [0 1])	plt.imshow(I, cmap="gray", vmin=0, vmax=1)
Afficher un gradient G	imshow(G, [])	plt.imshow(G, cmap="gray")
Disposer les images	subplot(1, n, k)	plt.subplot(1, n, k)
Ajouter un titre	title("Titre")	plt.title("Titre")
Définir le noyau	H = [1 1 1; 1 1 1; 1 1 1] / 9	H = np.ones((3, 3)) / 9
Appliquer un noyau	imfilter(I, H, 'replicate')	cv2.filter2D(I, -1, H, borderType=cv2.BORDER_REPLICATE)
Bruit sel et poivre	imnoise(I, 'salt & pepper', 0.05)	util.random_noise(I, mode='s&p', amount=0.05)
Filtre médian	medfilt2(Ib, [3 3])	ndimage.median_filter(Ib, size=3)
Norme du gradient Sobel	imgradient(I, 'sobel')	filters.sobel(I)
Norme du gradient Prewitt	imgradient(I, 'prewitt')	filters.prewitt(I)
Canny	edge(I, 'canny')	feature.canny(I)

Modules nécessaires

Python : np = numpy ; plt = matplotlib.pyplot ; cv2 = OpenCV ; ndimage vient de scipy ; img_as_float, util, filters et feature viennent de skimage. Sous MATLAB, utiliser Image Processing Toolbox.